

# File map & implementation order

**10 first-run files + 2 deferred files.** Work inside cuboid\_translation\_x/. Edit the two existing files; create the others. Reuse physics\_formula.py and labels/ as dependencies.

**Authority:** experiment rules below follow the PDF. File boundaries, API names and stacks are implementation recommendations. Unspecified numerical choices remain open.

| Order / file                | Responsibility                                   |
|-----------------------------|--------------------------------------------------|
| 01 config.py                | Fixed protocol; unresolved run choices; seed     |
| 02 data.py                  | CSV loading; IDs; reference groups; scaling      |
| 03 neural_network.py [edit] | 4-input / 2-output model                         |
| 04 metrics.py               | Full-set evaluation; rulers; groups; baselines   |
| 05 checkpoints.py           | Snapshots; absolute best; reload verification    |
| 06 stopping.py              | Target / progress / budget decisions             |
| 07 checks.py                | Data, scaling and learning preflight checks      |
| 08 training_loop.py [edit]  | Batch updates; evaluations; logging; termination |
| 09 report.py                | Curves; failures; summary; diagnosis             |
| 10 final_test.py            | Frozen-choice test evaluation; separate results  |
| 11 animate.py [deferred]    | First-model demonstration after result review    |
| 12 compare.py [deferred]    | Controlled comparisons; separate rollout study   |

## START CHECKLIST

**Open choices:** stopping ruler = coarse / intermediate / fine; absolute min\_delta in standardized validation-MSE units; record a seed. No invented defaults.

**Core stack:** Python standard library + NumPy + PyTorch + Matplotlib. Keep experiment settings in config.py; keep generated evidence in a separate run directory.

**Read downward:** each numbered box is one coding task. Complete its CHECK before following the arrow. Pages 2-5 continue the same sequence.

## 01-03 / Data contract & model

### 01 OPEN config.py / PROTOCOL

**Responsibility:** Keep the experiment settings in one place so every file uses the same rules, numerical choices and random seed.

**Stack / search:** Python dataclasses; pathlib; random.seed; numpy.random.seed; torch.manual\_seed.

**Write:** RunConfig. Inputs [v0, F, t1, t]; targets [d, v]. Train 8,000; validation 1,000; test 1,000 reserved. Fixed mass and friction. Network  $4 \rightarrow 32 \rightarrow 32 \rightarrow 2$ ; CPU; float32.

**Lock:** Adam; learning rate 0.001; weight decay 0; batch 128; maximum 10,000 updates; MSE averaged over standardized outputs and samples; patience 500 updates; coverage  $\geq 95\%$  on BOTH full train and validation sets. No group gate.

**Rulers (d / v):** coarse 0.1 m / 0.1 m/s; intermediate 0.01 m / 0.01 m/s; fine 0.001 m / 0.001 m/s. Select ONE for stopping; evaluate all three throughout.

**CHECK:** ruler, min\_delta and seed recorded before the run. Progress clock starts at update 0; no warm-up.



### 02 OPEN data.py / LOAD → GROUP → SCALE

**Responsibility:** Turn CSV rows into checked, scaled inputs and targets for the model. Keep row IDs and motion groups attached so predictions can be traced and compared.

**Stack / search:** csv.DictReader; NumPy arrays; hashlib.sha256; torch.utils.data.Dataset / DataLoader.

**Write:** load\_split(), attach\_groups(), fit\_scaler(), transform(), inverse\_transform(). Map CSV columns to [v0, F, t1, t] and [d, v]; retain split + row ID. Check columns, finite values, valid times and shapes.

**Reference groups:** always resting:  $v_0 = 0$  AND  $\text{abs}(F) \leq \mu_s \times m \times g$ ; moved then stopped: not always resting AND reference final  $v = 0$ ; moving at observation: reference final  $v \neq 0$ , including reversals.

**Keep:** exact reference rest/stop logic; disjoint, exhaustive groups; IDs and groups aligned through shuffling. Groups are metadata, never inputs or targets. Do not infer movement from net displacement. Existing CSVs need no overwrite.

**Scale:** fit on training rows only; handle near-zero scales; record the scale-handling choice. CHECK inverse recovery, [N,4] inputs / [N,2] targets, and rest/sliding/stopping/reversal/breakaway labels.



### 03 OPEN neural\_network.py / PREDICT

**Responsibility:** Define the trainable network that maps initial conditions and time to predicted displacement and velocity, using standardized inputs and outputs.

**Stack / search:** PyTorch nn.Module; nn.Linear; nn.Sequential; nn.ReLU; nn.MSELoss.

**Write:** retain transmodel; linear widths  $4 \rightarrow 32 \rightarrow 32 \rightarrow 2$ . Recommendation: retain the existing hidden ReLU layers; the PDF does not specify activation. Inputs and outputs use the recorded normalization.

**CHECK:** CPU float32; batch output [N,2]; finite predictions. Inverse-scale outputs before physical errors or tolerance comparisons.

## 04-06 / Evaluation & stopping

### 04 OPEN metrics.py / MEASURE

**Responsibility:** Measure how accurate predictions are: calculate loss, physical errors and pass rates for whole datasets and motion groups, and compare simple baseline guesses.

**Stack / search:** torch.inference\_mode; model.eval; reduction="none"; NumPy percentile; Boolean masks.

**Write:** evaluate(), group\_metrics(), baseline\_metrics(). Evaluate full train and validation with the SAME unchanged model. Standardized MSE = sum of the two squared output errors over rows / (2 × row count). Aggregate by sample counts, never average batch means.

**Joint pass:**  $\text{abs}(d\_error) < \tau_d$  AND  $\text{abs}(v\_error) < \tau_v$ . Strict inequalities. Compute all three rates from the same predictions. Keep per-row squared errors before group aggregation.

**Diagnostics:** full-validation group counts, MSE, all joint pass rates; physical d/v MAE, P95 and maximum errors. Report overall physical errors too. Empty group: count 0, metrics undefined.

**Extra slice:**  $v_0 = 0$  AND  $\text{abs}(\text{abs}(F) - 3.924\text{ N}) \leq 0.2\text{ N}$ ; overlaps groups; no acceptance gate. At update 0 score [0,0] and  $[v_0 \times t, v_0]$  baselines. Keep per-row errors at updates 0 and 10, if reached.



### 05 OPEN checkpoints.py / PRESERVE

**Responsibility:** Save and reload model snapshots and training state so results can be reproduced, the best model recovered and a run continued.

**Stack / search:** torch.save / torch.load; state\_dict; Python json; RNG getstate / setstate.

**Write:** save\_checkpoint(), load\_checkpoint(), save\_if\_best(). Save every scheduled evaluation, final and selected models. Store weights, model config and normalization; optimizer and RNG states for continuation. Recommend recording update, epoch and sampler position for continuation.

**Absolute best:** save every new validation-MSE minimum, even an improvement smaller than min\_delta. Keep this best separate from the progress reference. CHECK reload reproduces predictions.



### 06 OPEN stopping.py / DECIDE

**Responsibility:** Decide when training should stop by checking accuracy targets, lack of meaningful progress and the update limit; return the reason and model-selection decision.

**Stack / search:** Python dataclass; Enum; state machine. Write StopState and check\_stop(); return decisions to the checkpoint writer.

**State:** progress-reference MSE and update; separate absolute-best MSE/checkpoint. Initialize progress reference at update 0. Inputs: update, matched overall train/validation selected-ruler rates, validation MSE.

**Exact check order:** finite metrics → save new absolute best → target → update progress reference → patience → cap. Numerical failure aborts separately; never success.

**Target:** both rates ≥95%, including update 0: TARGET\_REACHED; save/select the first-passing checkpoint. Otherwise meaningful progress iff  $\text{current\_val\_mse} < \text{reference\_mse} - \text{min\_delta}$ ; replace reference and reference update only then.

**Stop:** elapsed updates since reference ≥500: NO\_PROGRESS. At 10,000: MAX\_UPDATES. If both, record both flags; primary NO\_PROGRESS. Select lowest-validation-MSE checkpoint unless target reached. Small gains accumulate against the unchanged reference; stops occur only at checks.

## 07-09 / Check, train & diagnose

### 07 OPEN checks.py / PREFLIGHT

**Responsibility:** Catch mistakes in data, scaling and learning before the real run, using small checks that confirm the training pipeline works.

**Stack / search:** Python unittest; numpy.testing.assert\_allclose; torch.isfinite; tensor.clone.

**Write:** check\_data(), check\_scaling(), check\_learning(). Check data and reference cases; training-only scaler and inverse recovery; one update changes weights with finite loss/gradients; substantially reduce error on 32-64 fixed examples.

**CHECK:** small fit is an implementation check, not generalization evidence or a perfect-boundary-fit requirement. Start fresh afterward. Recommended stop tests: target at 0; strict tolerance boundary; sub-min\_delta accumulation; progress at 200 then stop at 700; patience + cap; numerical abort.



### 08 OPEN training\_loop.py / ORCHESTRATE

**Responsibility:** Run the experiment: feed batches to the model, update its weights, schedule evaluation, save checkpoints, log evidence and apply stopping decisions.

**Stack / search:** torch.optim.Adam; DataLoader(shuffle=True, drop\_last=False); time.perf\_counter; csv.DictWriter.

**Setup:** seed → load train/validation → fit scaler → fresh model + Adam → evaluate update 0 → checkpoint + stop check. Keep test reserved.

**Update:** batch → predict → MSE → clear gradients → backward → optimizer update → increment global step. Shuffle each epoch; keep partial batch and already-passing samples. Record batch loss each update; abort numerical failures separately.

**Evaluate:** updates 0, 10, 20, 50, 100; then every 100; also normal termination. Use full matched train/validation metrics, never the latest batch pass rate. Invoke the ordered checkpoint/stop pipeline; restore train mode if continuing.

**CSV:** update, epoch, sample exposures, elapsed time, split, MSE and three pass rates; group metrics/counts; physical errors; breakaway slice. Retain raw observations. Count actual rows seen, including partial batches.

**CHECK:** first possible stagnation stop = update 500; evaluate and stop at 10,000; never run update 10,001. At exit preserve final, selected and last valid evidence; record reason and all stop flags.



### 09 OPEN report.py / CURVES → FAILURES → REVIEW

**Responsibility:** Turn saved results into plots, failure tables and a readable summary so you can see what the model learned, where it struggles and what to investigate next.

**Stack / search:** Matplotlib pyplot.subplots / Axes.plot; csv; NumPy percentile; json. Write plot\_curves(), failure\_table(), summarize\_run().

**Plots:** validation MSE and pass rates versus updates; matched train/validation curves; full-validation group curves with counts. Pass-rate axis 0-100%. Preserve raw trends; require neither a logarithmic shape nor a plateau.

**Failure tables:** selected AND last valid checkpoint; row ID, input, reference group, target, prediction, physical errors, severity = max(abs(d\_error)/tau\_d, abs(v\_error)/tau\_v). Use selected ruler; keep physical units. Failed-only counts/errors; representative and worst cases. Zero failures: count 0, failed-only statistics undefined.

**Run record:** config, data hashes/row IDs, seed, software/device, normalization, updates/exposures/time, termination. Summary: ruler/coverage, measured rates, groups, physical errors, baselines, reason and limits.

**Review:** raw trend + stop settings → matched train/validation → failing groups/examples → one explanation + one controlled follow-up. Use validation; no automatic model enlargement or longer training. Timing is supporting context.

## 10-12 / Freeze, demonstrate & compare

### 10 OPEN final\_test.py / FROZEN CHOICE ONLY

**Responsibility:** Measure the frozen, selected model on reserved test data and save separate results to assess performance on examples unused in model selection.

**Stack / search:** reuse data.py, metrics.py and checkpoints.py; Python json; torch.inference\_mode.

**Write:** evaluate\_frozen\_choice(). Freeze the training protocol and model selection BEFORE opening the reserved 1,000 test rows for evaluation. Reuse training-fitted normalization and the selected checkpoint.

**Report:** separate test metrics; no tuning against test results. Training target attainment is distinct from final test acceptance; the PDF supplies no separate numerical test-acceptance threshold.

**CHECK:** label the scope as synthetic one-dimensional reference agreement, not general 3D or real-world validation. If validation follow-ups are still changing the choice, defer this test execution.



### 11 OPEN animate.py / SECOND MILESTONE

**Responsibility:** Show the first trained model visually alongside the analytical motion reference, using saved checkpoints after the training result has been reviewed.

**Stack / search recommendation:** Matplotlib animation.FuncAnimation; saved PyTorch state\_dict; analytical reference.

**Gate:** review the first training result before animation design. Build the first-model demonstration before other experiments; Models B/C are not prerequisites.

**Write later:** load saved training checkpoints + recorded normalization; obtain corresponding analytical-reference predictions; implement the reviewed demonstration design.

**CHECK:** use the first model and its saved training progress. Animation design remains deferred as required by the PDF.



### 12 OPEN compare.py / AFTER THE DEMONSTRATION

**Responsibility:** Run fair follow-up experiments that change one main factor at a time, such as network size or training-data size, and compare the measured results.

**Stack / search:** Python dataclasses.replace; NumPy subset indexing; reuse training/evaluation/report modules; Matplotlib.

**Write later:** run\_architecture\_comparison(), run\_data\_comparison(), run\_loss\_comparison(), study\_rollouts(). Change one main factor at a time; preserve matched data and protocols.

**Architectures:** 4 → 16 → 2; 4 → 32 → 2; 4 → 16 → 16 → 2; 4 → 32 → 32 → 2. Repeat promising comparisons with three seeds.

**Data:** nested 500 / 1,000 / 2,000 / 4,000 / 8,000 training rows; fresh models; subset-specific normalization. Loss comparisons are deferred; candidate losses are not specified by this PDF.

**Rollouts:** separate study; matched force schedules; explicit position tracking; out-of-range checks. The PDF refers to supporting notes for comparison limitations; do not invent additional comparison standards in this manual.

**CHECK:** first-run training + evidence → result review → first-model animation → controlled comparisons. Keep final test evaluation tied to a frozen choice, not a recurring tuning loop.